



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Securing RAG Infrastructures:

Cryptographic Approaches for Encryption at Rest in Vector
Databases

Technical Report & Proof of Concept

Author: Nicolás Maire Bravo

Status: Erasmus+ Student

Email: nicolas.mairebravo@studio.unibo.it

Course: Project Work on Cybersecurity

Professor: Prof. Michele Colajanni

Institution: Alma Mater Studiorum - Università di Bologna

Academic Year: 2025/2026

Table of Contents

1. Introduction	3
2. Cryptographic Architecture	3
2.1. Mode of Operation: AES-GCM	3
3. Key Management: The Envelope Strategy	3
4. Proof of Concept (PoC)	4
4.1. The Setup	4
4.2. Execution Flow	4
4.3. Recovery and Validation	4
5. Conclusion	4

1. Introduction

Retrieval-Augmented Generation (RAG) systems have become a standard architecture for deploying Large Language Models (LLMs) in enterprise environments. These systems rely on vector databases to store embeddings of domain-specific documents, allowing the LLM to retrieve contextually relevant information.

A critical security challenge arises when the ingested documents contain Personally Identifiable Information (PII), such as names, social security numbers, or medical records. If the underlying storage infrastructure or the cloud buckets hosting the vector database are compromised, attackers can gain unauthorized access to sensitive data, leading to severe privacy breaches.

While pre-vectorization masking and tokenization are valid strategies, a defense-in-depth approach requires securing the physical storage layer. This report proposes an architecture focused on Encryption at Rest, ensuring that even if physical drives or raw database files are exfiltrated, the PII remains cryptographically secure and inaccessible without the proper authorization keys.

2. Cryptographic Architecture

To mitigate physical data breaches, Encryption at Rest is implemented as the primary security control for the vector storage layer. The core of this implementation relies on symmetric block ciphers, specifically the Advanced Encryption Standard (AES) with a 256-bit key length.

2.1. Mode of Operation: AES-GCM

Applying a block cipher requires selecting an appropriate mode of operation. For database storage, traditional modes like Electronic Codebook (ECB) fail to hide data patterns, and Cipher Block Chaining (CBC) lacks built-in data integrity verification.

This architecture implements AES in Galois/Counter Mode (AES-GCM). AES-GCM is an Authenticated Encryption with Associated Data (AEAD) cipher. It provides two essential security properties for the RAG infrastructure:

- **Confidentiality:** The vector embeddings and associated metadata are encrypted using a counter-driven stream cipher approach.
- **Integrity and Authenticity:** GCM calculates an authentication tag (MAC) using universal hashing over a binary Galois field.

By verifying the authentication tag during the decryption phase, the system can detect any unauthorized modification to the database files. This prevents tampering attacks where an adversary might attempt to subtly alter the stored vectors to manipulate the LLM's retrieved context.

3. Key Management: The Envelope Strategy

Encryption is useless if you leave the key under the doormat. That's the harsh reality of securing vector databases. You can lock down the cloud buckets with AES-256 all day long, but if the decryption keys sit right next to the data, a breach is inevitable.

To fix this, we don't just hide the key; we encrypt it. Enter Envelope Encryption. It sounds like a heavy cryptographic concept, but the mechanics are surprisingly straightforward. You encrypt your data with one key, and then you protect that key with another.

Here is the breakdown of the dual-key system powering this architecture:

- **Data Encryption Key (DEK):** A fast, symmetric key that does the heavy lifting. It encrypts the massive gigabytes of vector embeddings and raw PII.
- **Key Encryption Key (KEK):** The master key. It never touches the raw data. Its sole purpose in life is to wrap (encrypt) the DEK.

Think of the KEK as a bank vault and the DEK as a briefcase inside it. The Key Management Service (KMS) handles the KEK safely away from the database storage. If an attacker dumps the physical hard drives, they only get ciphertexts and a locked briefcase. Without hitting the KMS API to unwrap the DEK, the data remains pure cryptographic noise.

4. Proof of Concept (PoC)

Theory only goes so far. To prove the viability of this architecture, I built a lightweight Python simulation of a RAG storage pipeline. No bloated frameworks—just standard cryptography.hazmat primitives doing exactly what they are supposed to do.

4.1. The Setup

We start with a mock dataset (`pii_dataset.csv`) filled with generated, highly sensitive patient records (names, SSNs, medical notes). This represents the raw knowledge base right before it gets chunked and sent to the LLM's context window.

4.2. Execution Flow

The script kicks off by spinning up a simulated KMS. It generates a fresh 256-bit KEK. Then, the vector storage module requests a DEK to secure the incoming data.

Instead of passing keys in the clear, the KMS encrypts the DEK using AES-GCM and hands over the wrapped version. The storage module takes the plaintext DEK, locks down the CSV file, and immediately dumps the plaintext key from memory.

What ends up on disk? Just two binary files: `encrypted_database.bin` and `encrypted_dek.bin`.

4.3. Recovery and Validation

Decryption runs the tape in reverse. The system loads the wrapped DEK, asks the KMS to unwrap it using the master KEK, and finally uses the exposed DEK to parse the database back into readable text. The terminal output confirms a perfect match between the original data and the recovered payload. Zero data loss, and strict integrity validation thanks to the GCM tags.

5. Conclusion

Encryption at Rest is not a silver bullet for RAG security, but it is a non-negotiable baseline. Masking PII before vectorization handles application-layer leaks, but it does nothing if the underlying storage infrastructure is compromised.

The Envelope Encryption architecture demonstrated in this report bridges that gap. By leveraging AES-GCM, the system guarantees both confidentiality and integrity for the vector embeddings. More importantly, the dual-key strategy completely neutralizes the key distribution bottleneck. The heavy cryptographic lifting remains fast and local using the DEK, while the master KEK stays isolated and strictly controlled by the KMS.

Ultimately, this setup allows enterprises to deploy LLMs on sensitive datasets with confidence. If a physical drive is stolen or a cloud bucket is misconfigured, the blast radius is zero. The data is locked, the keys are elsewhere, and the privacy of the users remains intact.